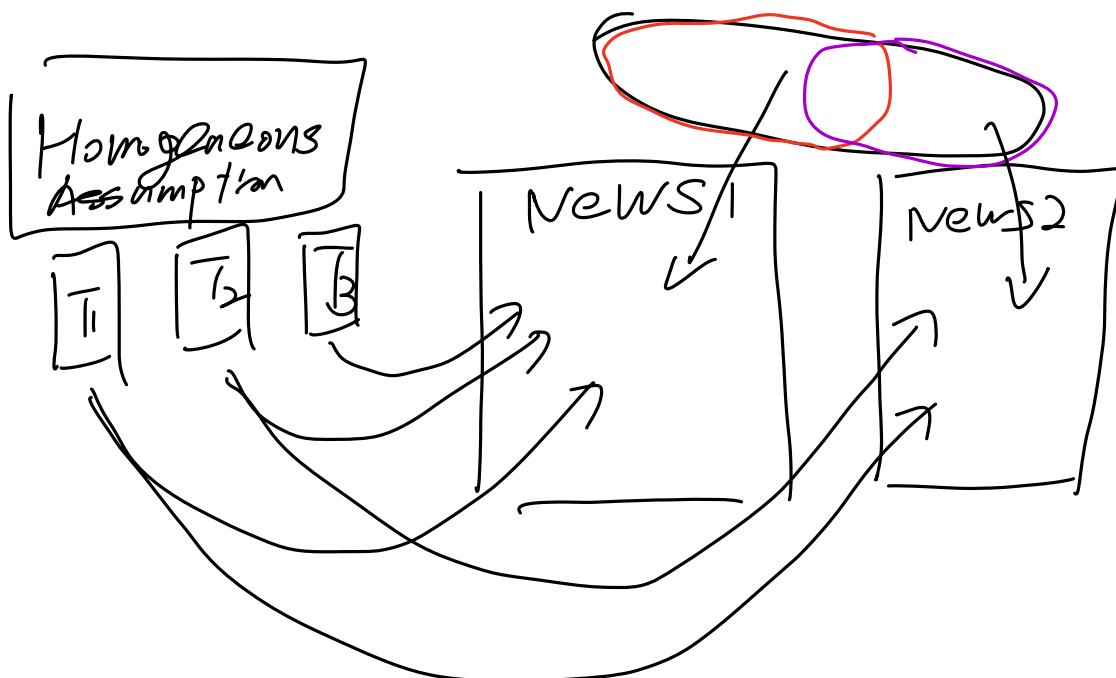
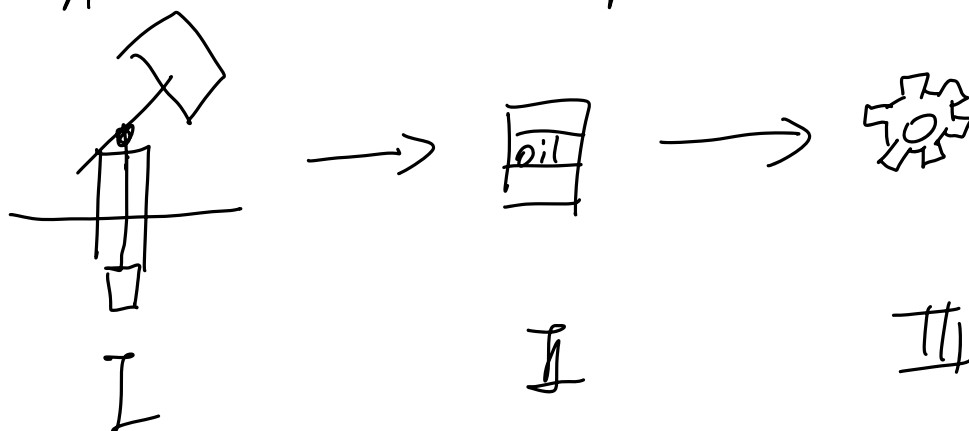




Bayesian classifier:

Need to Classify Good & Bad for inc A.

A is a crude oil production inc.



Text I

oil has release  
new tech on  
oil extraction

Text II

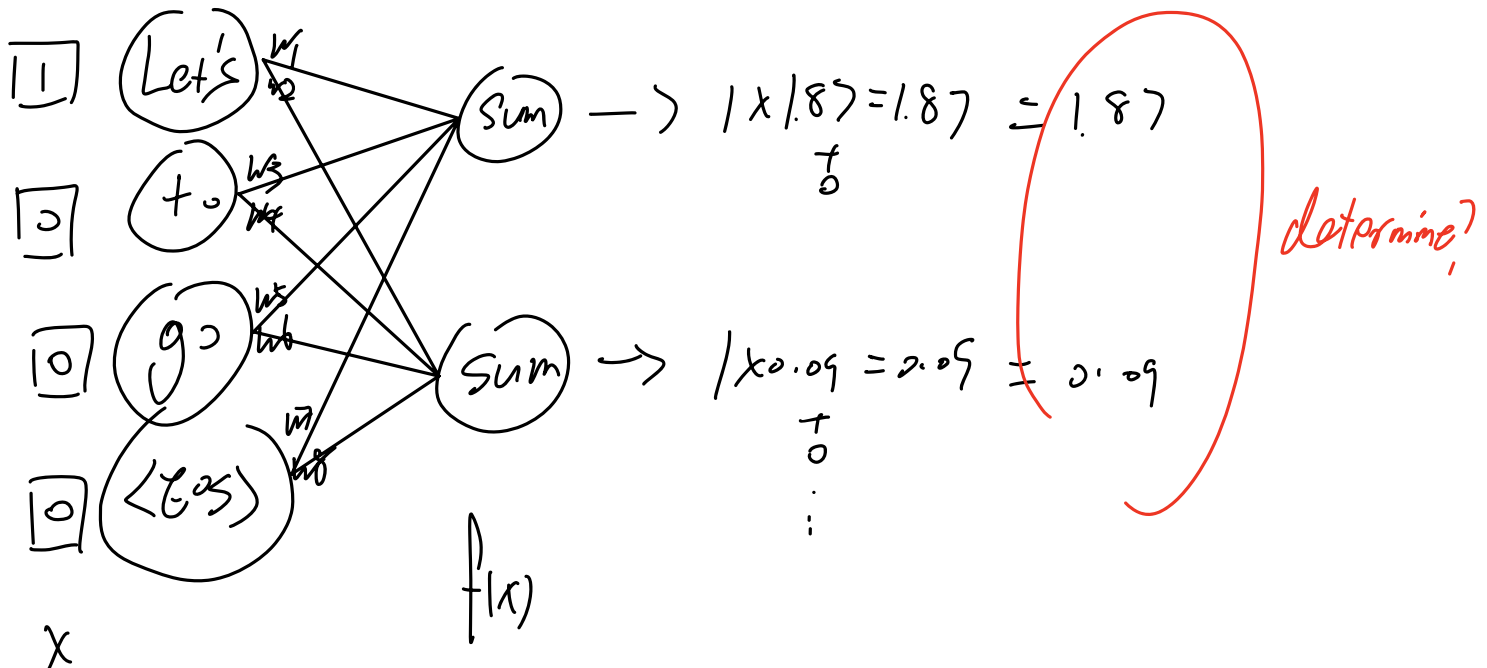
WB bought \$100M to  
oil Inc. ...

making it fast  
and high-quality

⇓  
★ Text Detection (NLP)  
Transformer

① Word Embedding

Text  $\rightarrow$  Number



Let's = (1.87, 0.09)

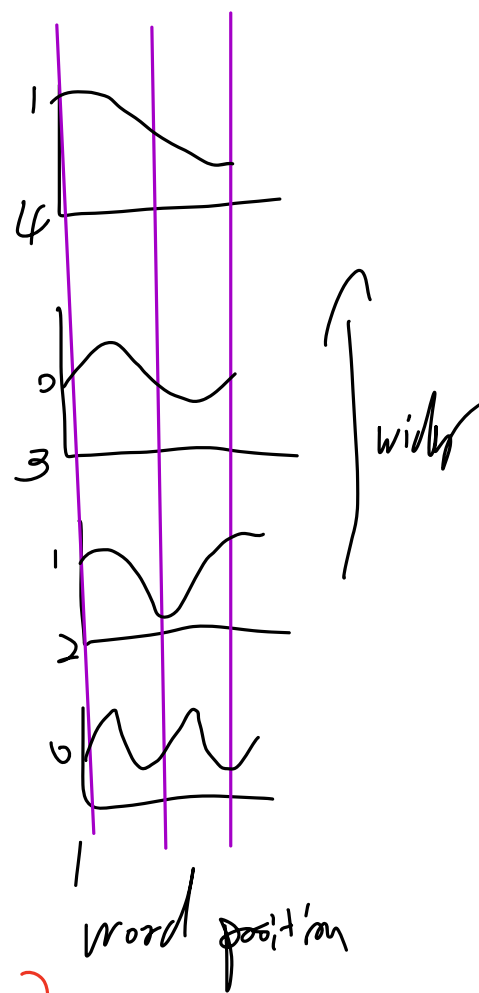
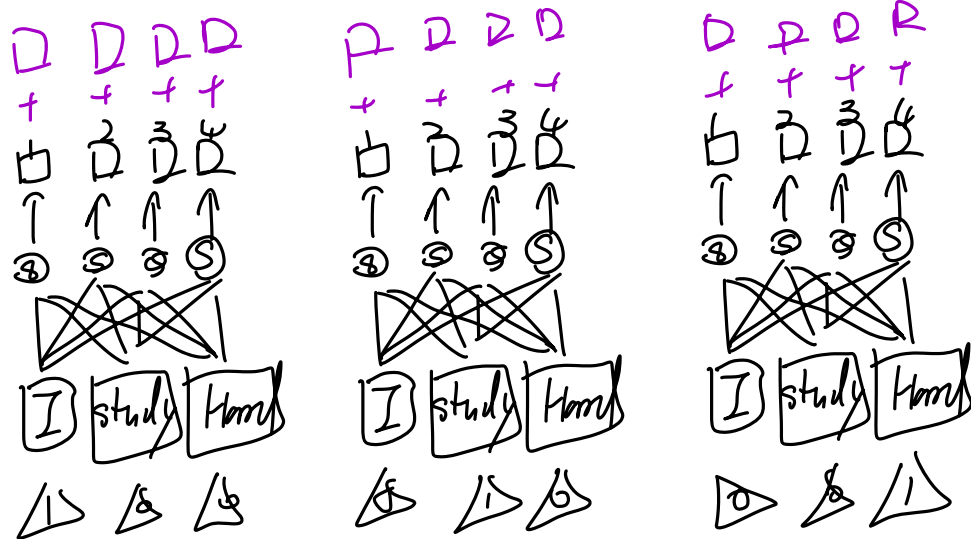
↑ same weights

↓

$$\textcircled{90} = (-0.78, 0.27)$$

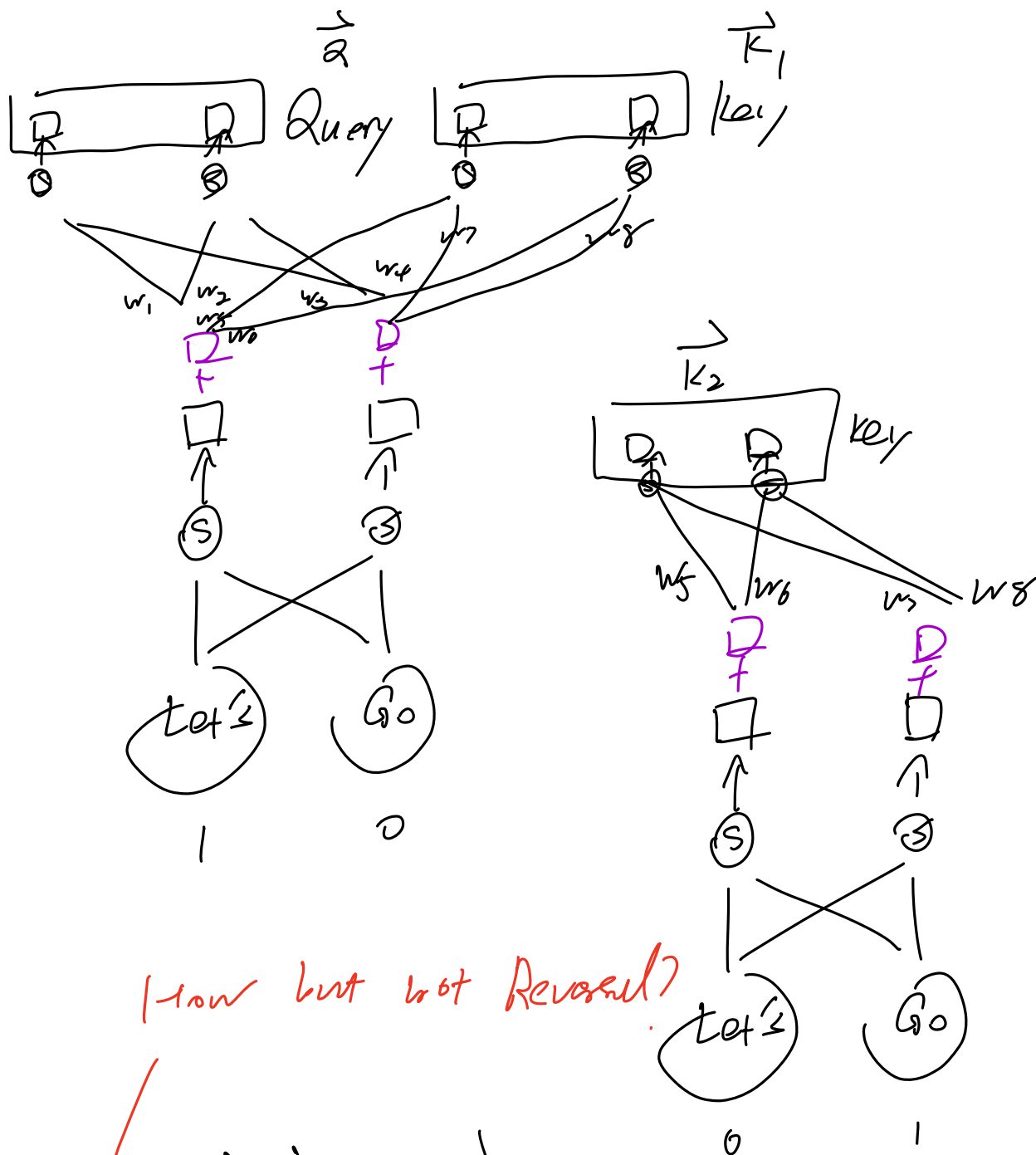
weights determined by Backpropagation

## ② Positional Encoding



⇒ unique as word embed more + wider

### ③ Self-Attention



How but not Reversed?

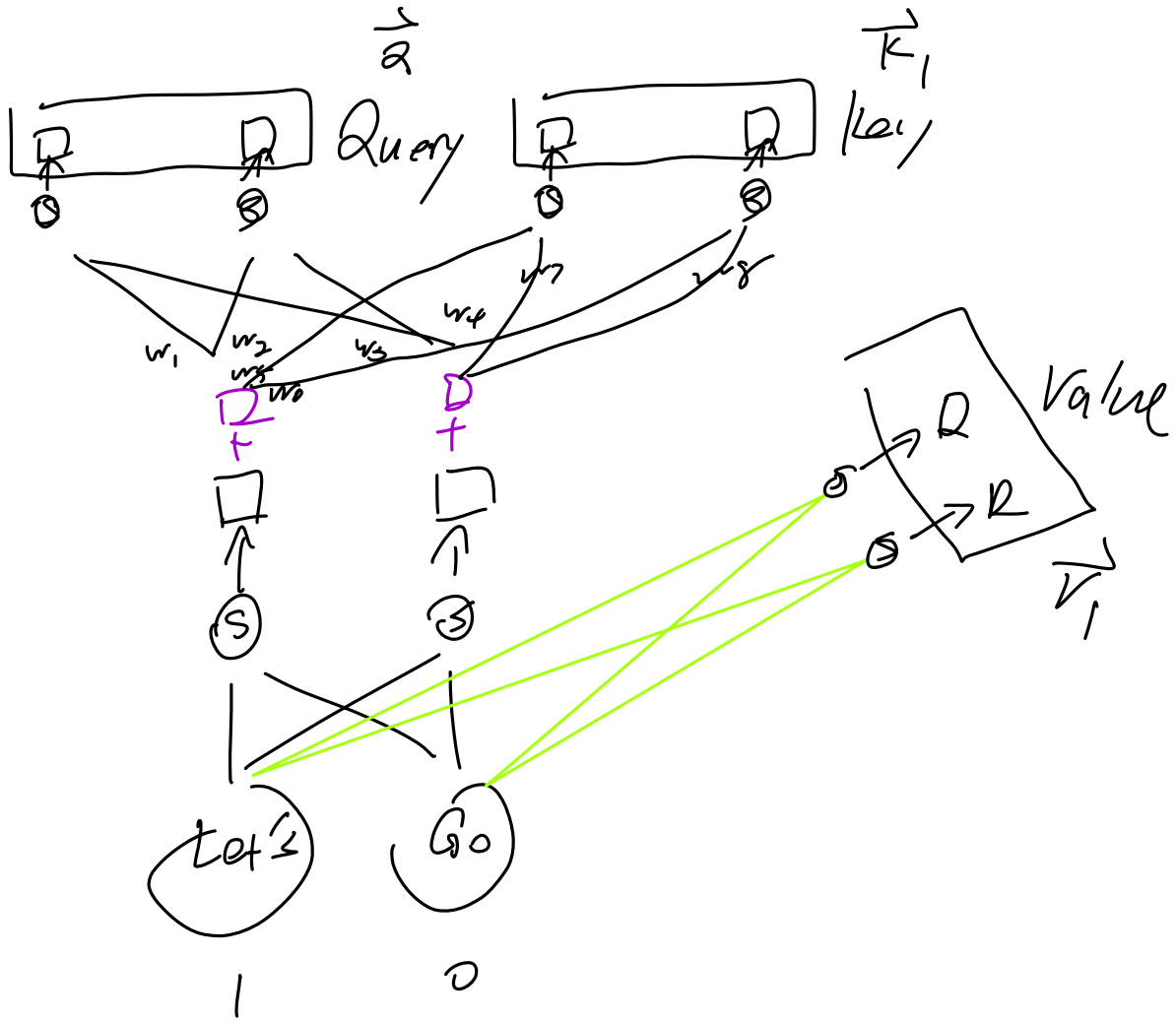
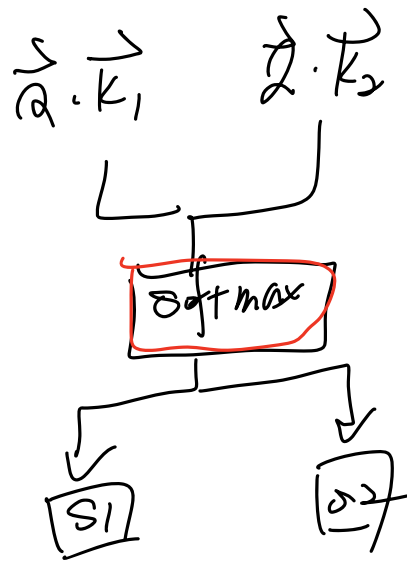
$$\vec{Q} \cdot \vec{K}_1 \quad \vec{Q} \cdot \vec{K}_2$$

$$11.7$$

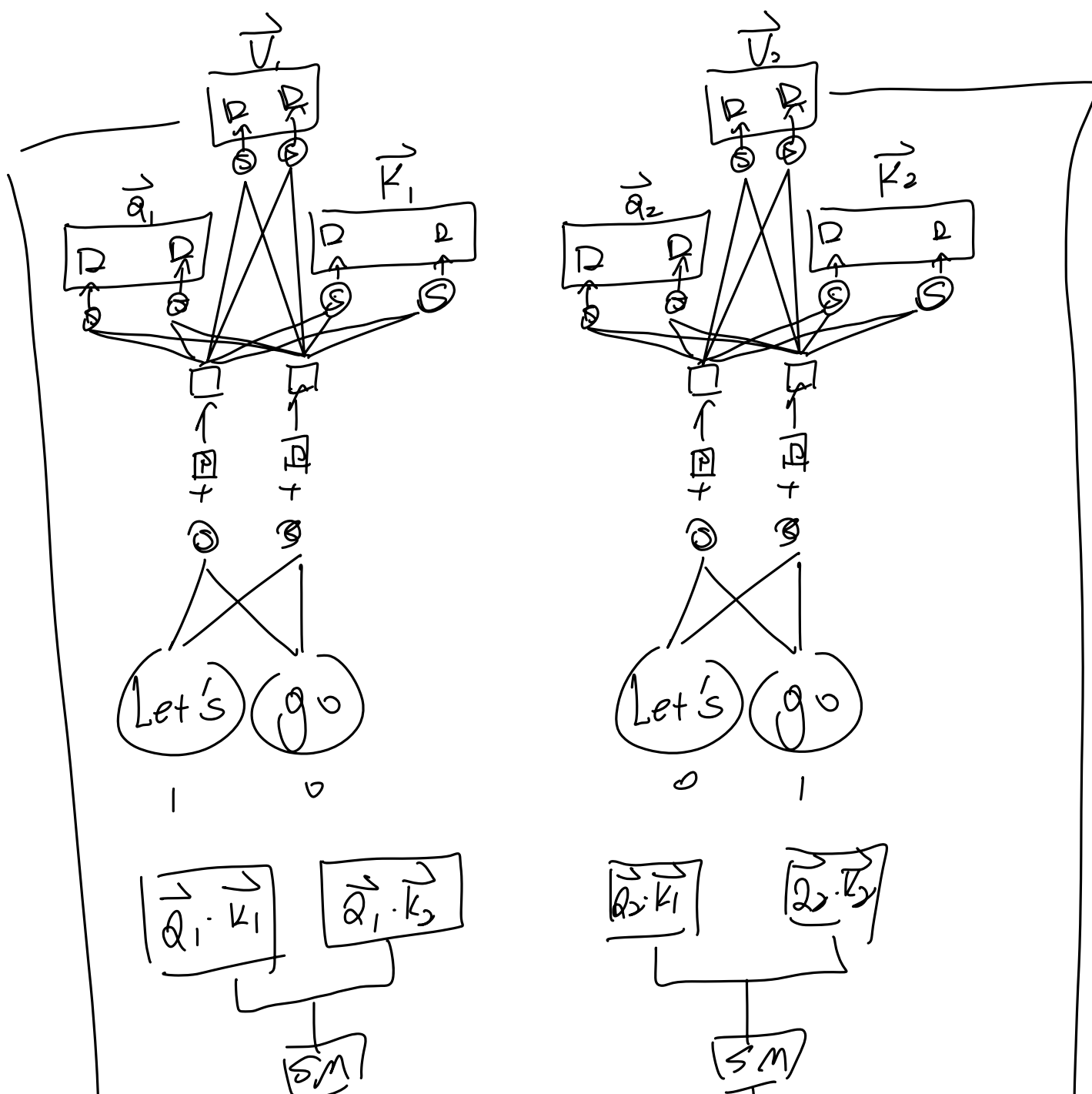
$$-2.6$$

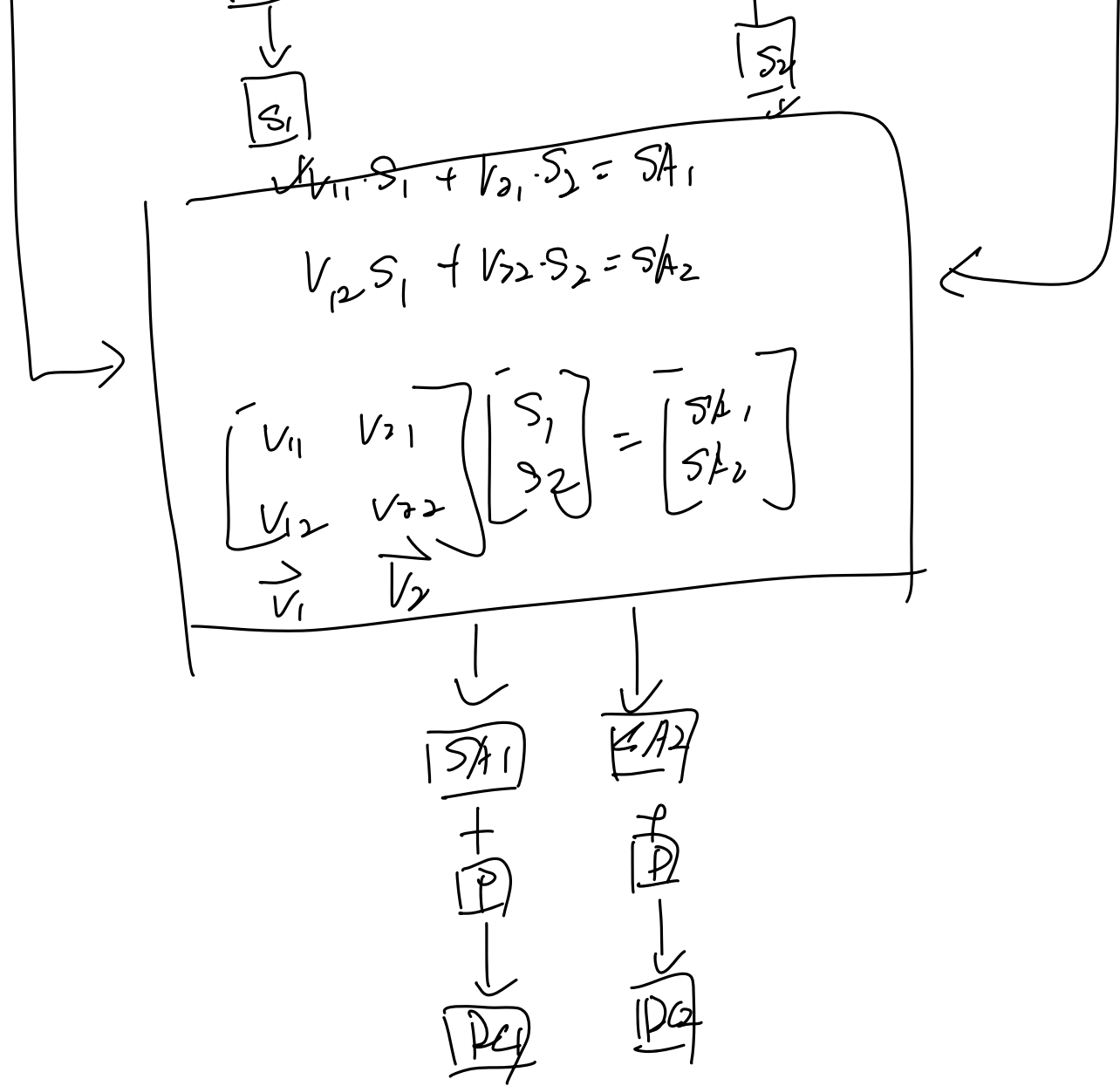
$\Rightarrow$  Let's  $\leadsto$  Let's

I sell the stock since it shows bad performance



$$\begin{aligned}
 & \vec{V}_{11} \cdot S_1 & \vec{V}_{12} \cdot S_1 \\
 & + \vec{V}_{21} \cdot S_2 & \vec{V}_{22} \cdot S_2 \\
 & = S_1^* & = S_2^* = \text{self attention value} \\
 & & \text{for } \text{Let}'_3
 \end{aligned}$$



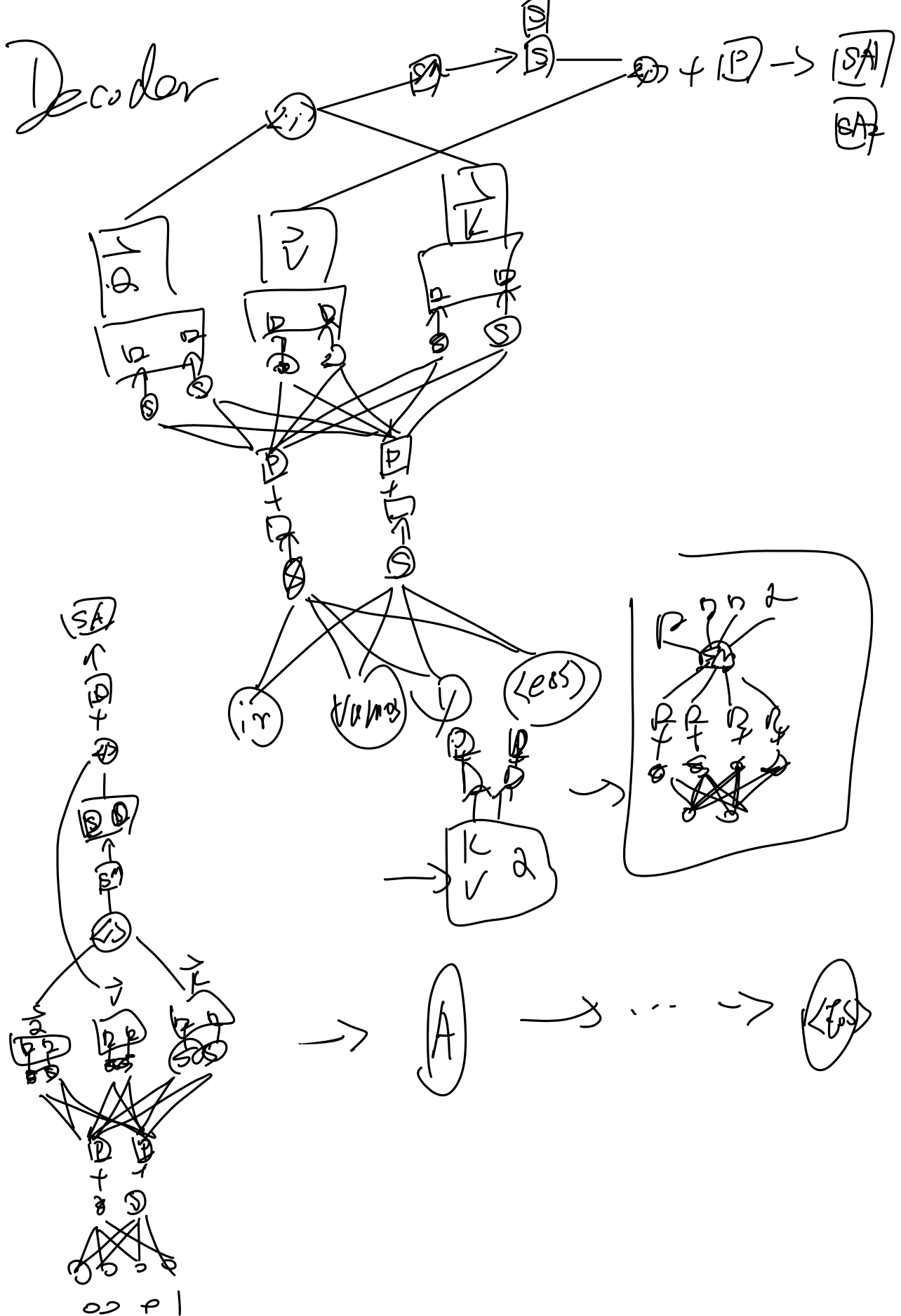


Word Embedding  $\rightarrow$  Positional Encoding  $\rightarrow$

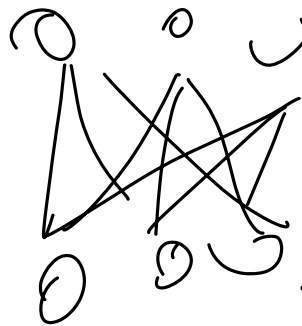
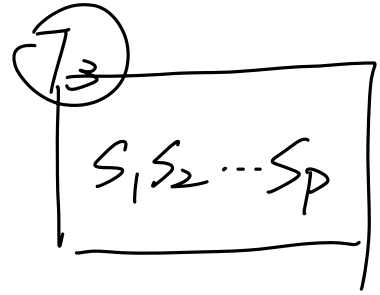
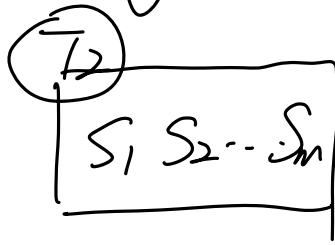
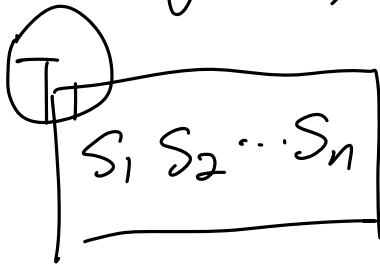
Self-Attention  $\rightarrow$  Residual Connection

Encoder

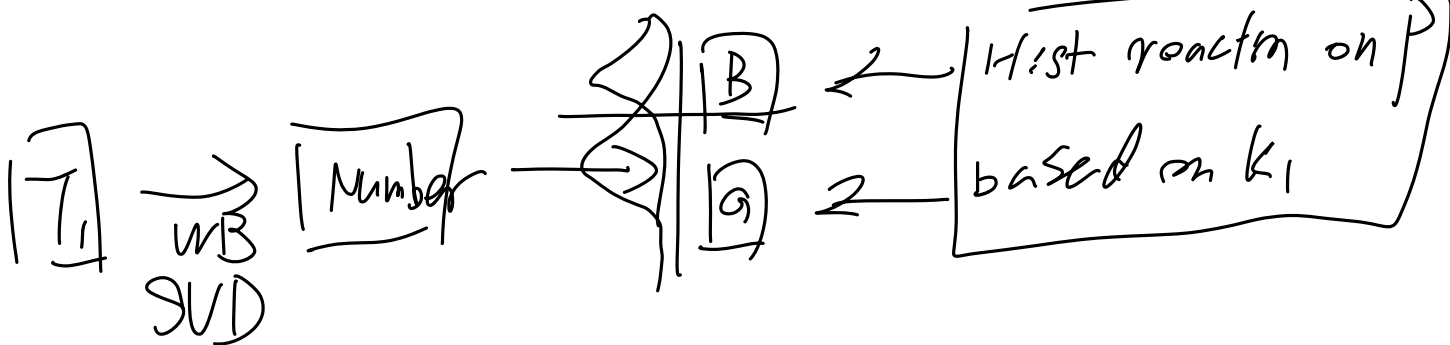
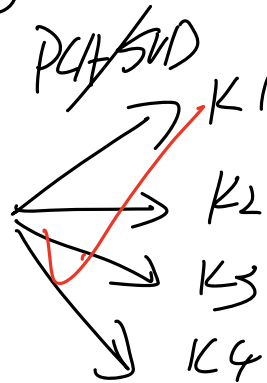
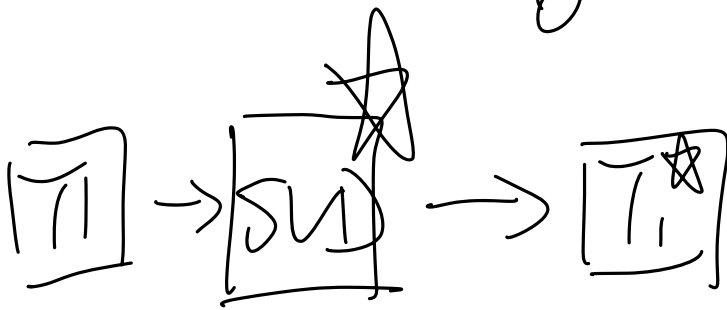
# Decoder



Homogeneous / Non homogeneous



one hot word 2 vec



to make sure stochastically uniform in t

$$P(t, k_1, k_2)$$

Why Res min?

★ Homogeneous / Non homogeneous

Define  $T: \mathcal{S} \rightarrow V$

$\mathcal{S}$  is sentence space

$V$  is normal vector space (Euclidean Norm)

Then  $T(s_1) = v_1 \dots T(s_n) = v_n$

$T$  is the same as word Embedding

so  $T(\cdot) \Leftrightarrow WB(\cdot)$

Similarly,

we have  $P(\cdot)$ ,  $SA(\cdot)$  as positional Encoding  
and Self-Attention

$$\Rightarrow TEncoder(\cdot) = SA \circ P \circ T^n$$

↑ similarity among words  
↑ order info  
← WB

Note word2vec can be referenced.

Define Coordinator C

$$\vec{K}_t = \text{svd}(\text{TEncoder}(\tau))$$

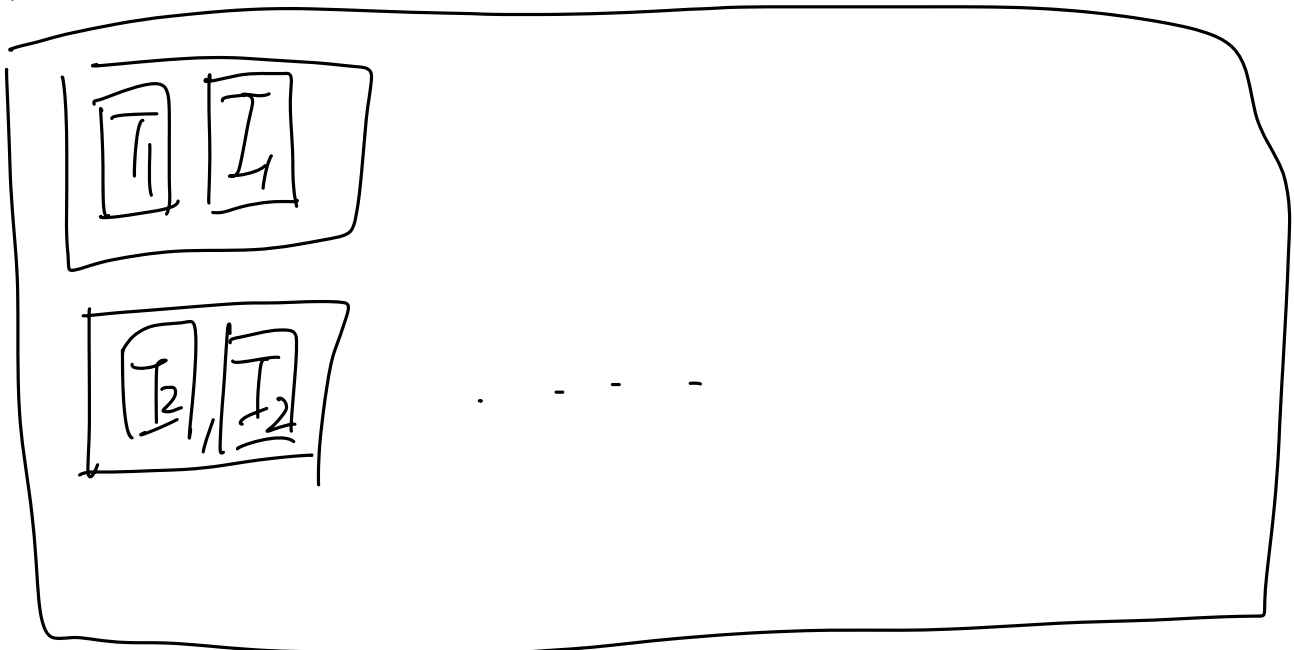
$$C(\vec{K}_t) = \hat{p}(t)$$

$$\Rightarrow p(L_t, \Phi(t))$$

---

★ Influence Prob  
Determine a set of data with cons.  
to learn weights to train loss function

Data =

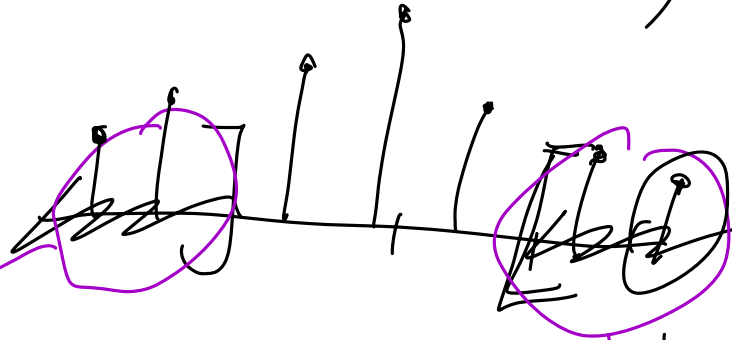


$$\overline{I} = T_{\text{Encoder}}(\text{Text})$$

$$(I_1, I_2, \dots) \xrightarrow{\text{Normalized}} (i_1, i_2, i_3, \dots)$$

$$\overline{I}$$

$\longleftrightarrow$



$$\text{svd}(\overline{I}) = (k_1, k_2, k_3, \dots)$$

$$\max \text{svd}(\overline{I})$$

$$\text{s.t. } (i_1, i_2, i_3, \dots)$$

$$\min \text{svd}(\overline{I})$$

$$\text{s.t. } (i'_1, i'_2, \dots)$$

max sub(T<sub>N</sub>)  
s.t. i<sub>N</sub>



$\{w_{N,i}\}$

max sub(T<sub>N-1</sub>)

s.t. i<sub>N-1</sub>



$\{w_{N-1,i}\}$



$$T_{\text{node}}(T) = \boxed{\boxed{t}}$$

$\vec{w}_N$

$\vec{w}_{N-1}$

$\vec{w}_{N-2}$

...

$\vec{w}_1$

$\begin{bmatrix} w_{N,1} \\ \vdots \end{bmatrix}$



$\begin{bmatrix} \end{bmatrix}$



$\begin{bmatrix} \end{bmatrix}$



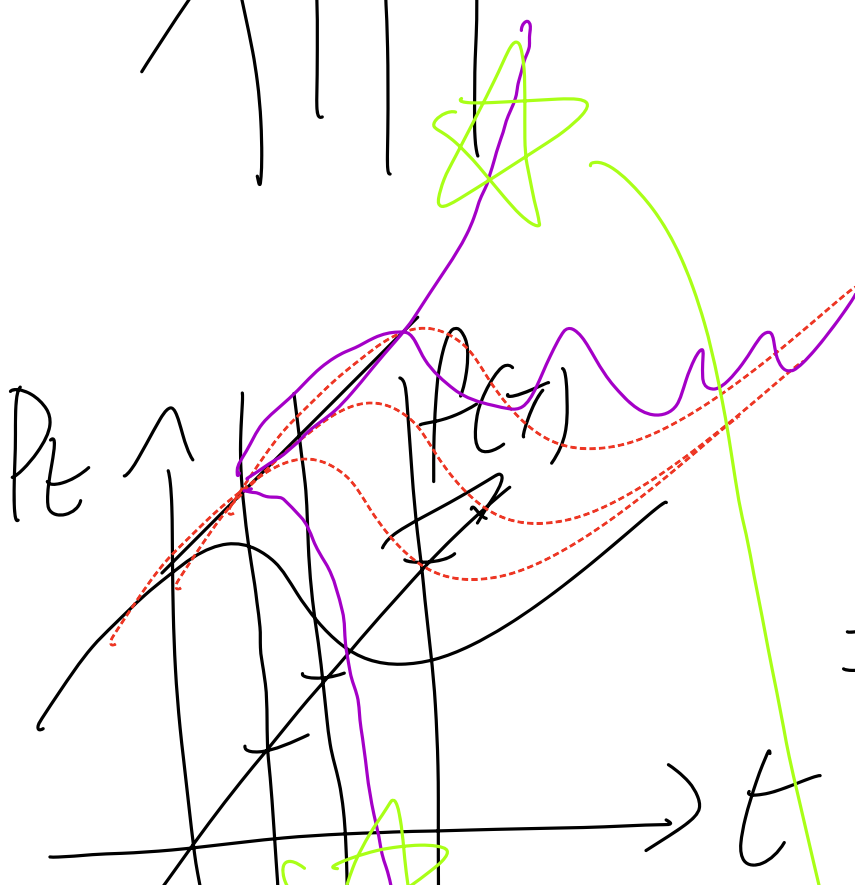
$\begin{bmatrix} \end{bmatrix}$

Interpolation

$[T] \rightarrow [F(t)]$  Reaction Mechanism

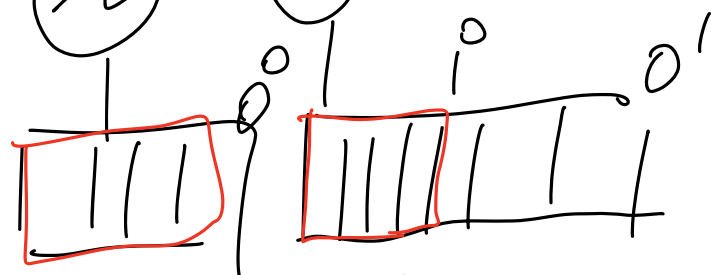
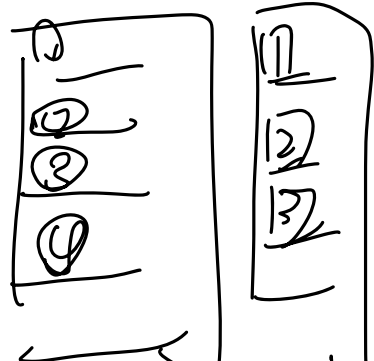
correspondence build

(I)



Realize

$\Rightarrow \{ \{ a_i \} \}$

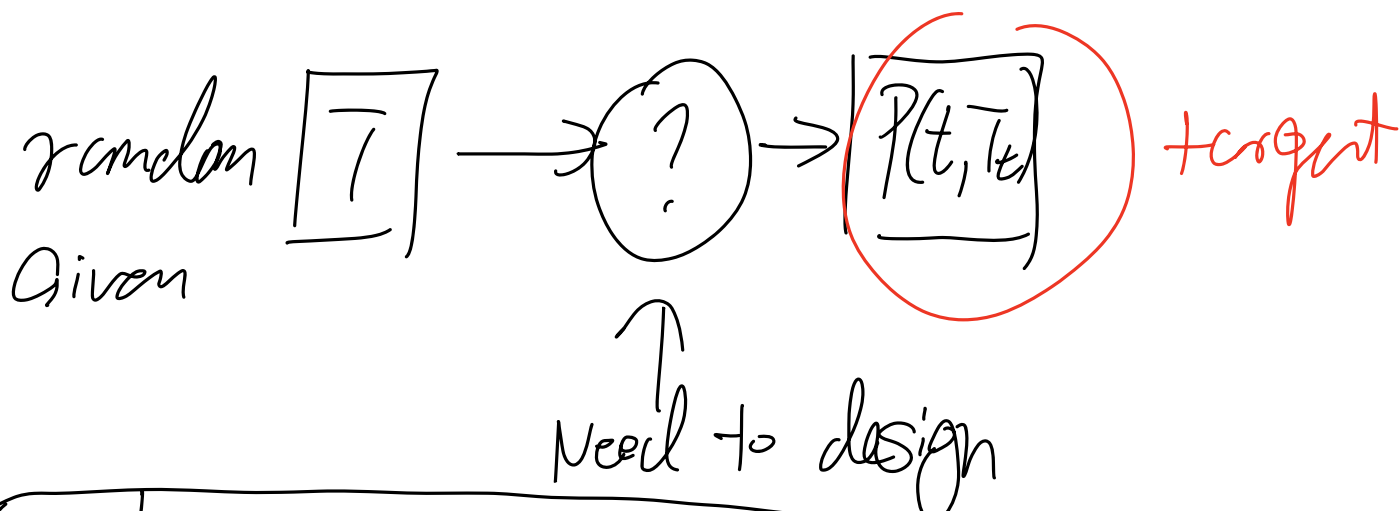


$$f_{CT,4} \in (\mathbb{R}^{384}, \mathbb{N}^t)$$

$$\{v_i^*\}$$

$$\{ \{v_i^*\} \}$$

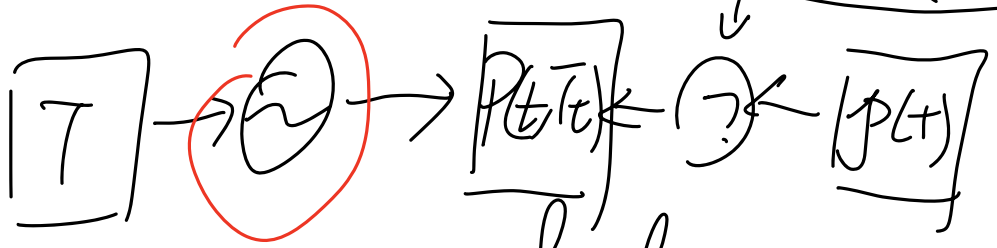
Revisit Influence Problem



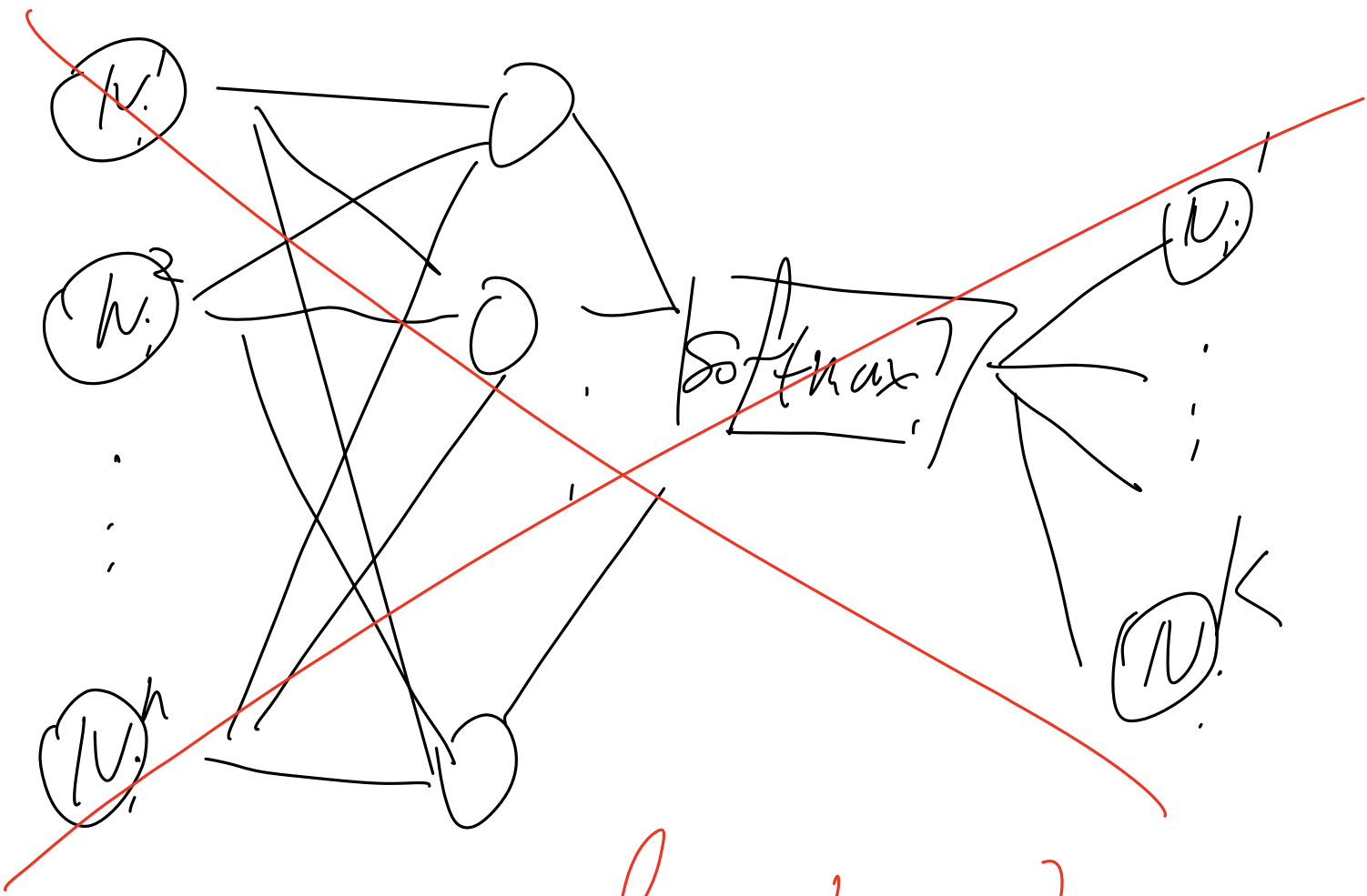
Remark:

PLMN of Inverse

Backward Equation



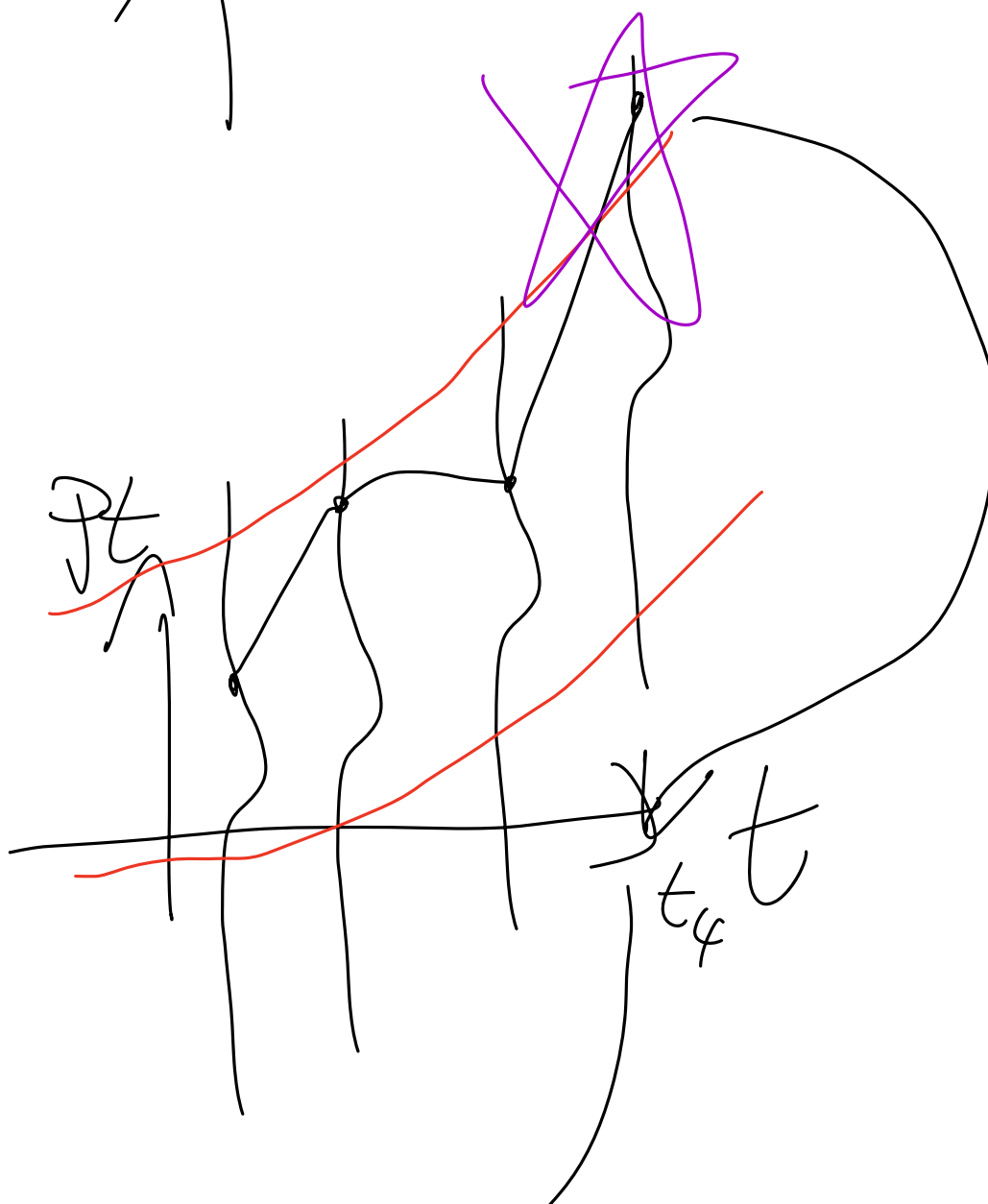
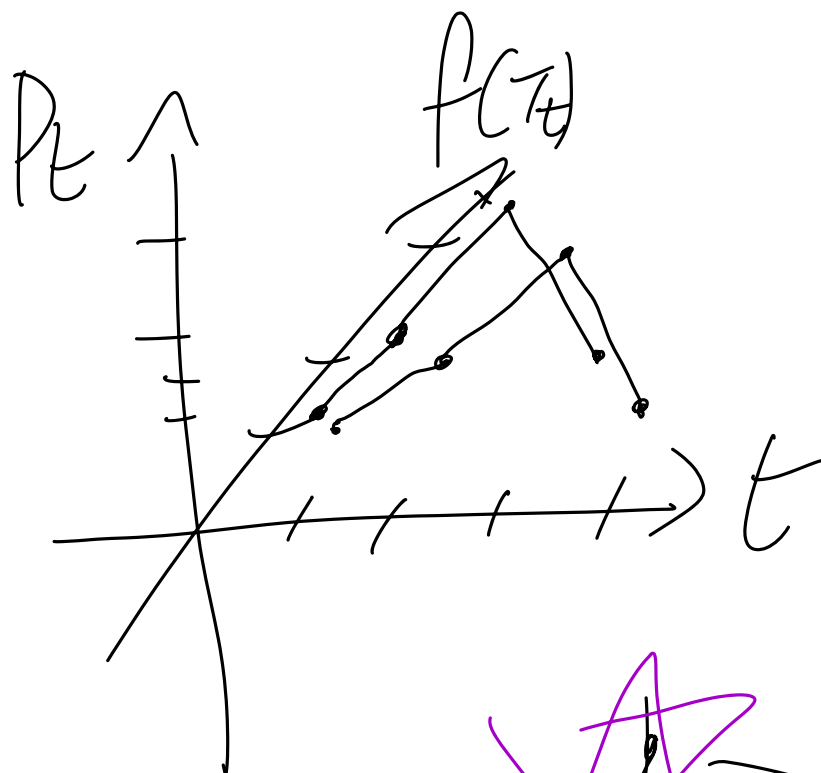
Given  $\rightarrow$  ~~target~~  $\leftarrow$  Inferal  $\leftarrow$  Given



Solution: Unified NN?

II

Detection method



$$T(T_3, T_4)$$

SVU Explain

July 29th

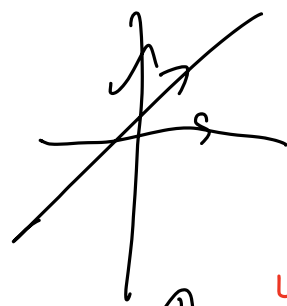
$U^T$

$m \times n$

$m \times m$

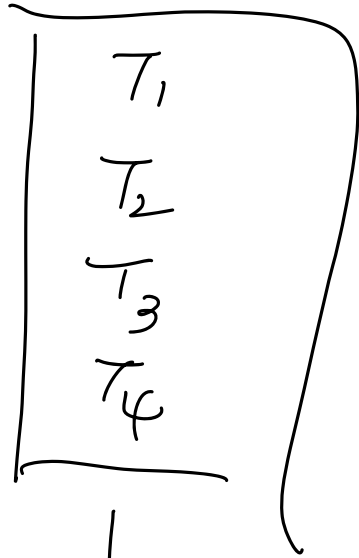
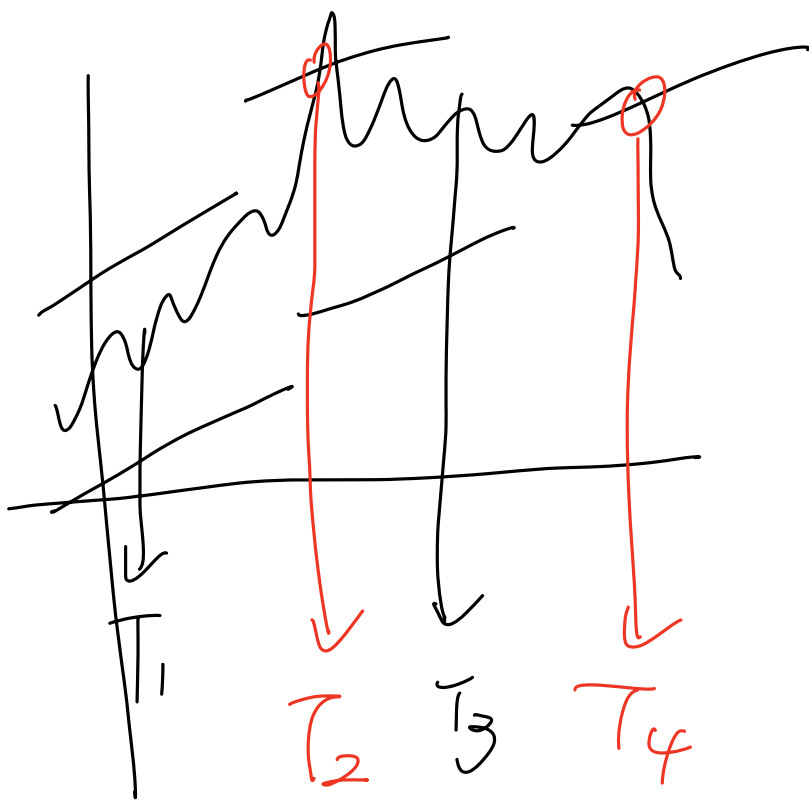
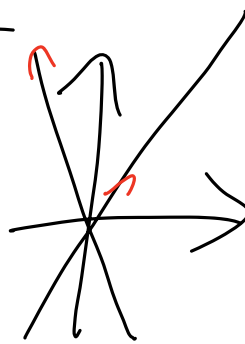
$n \times n$

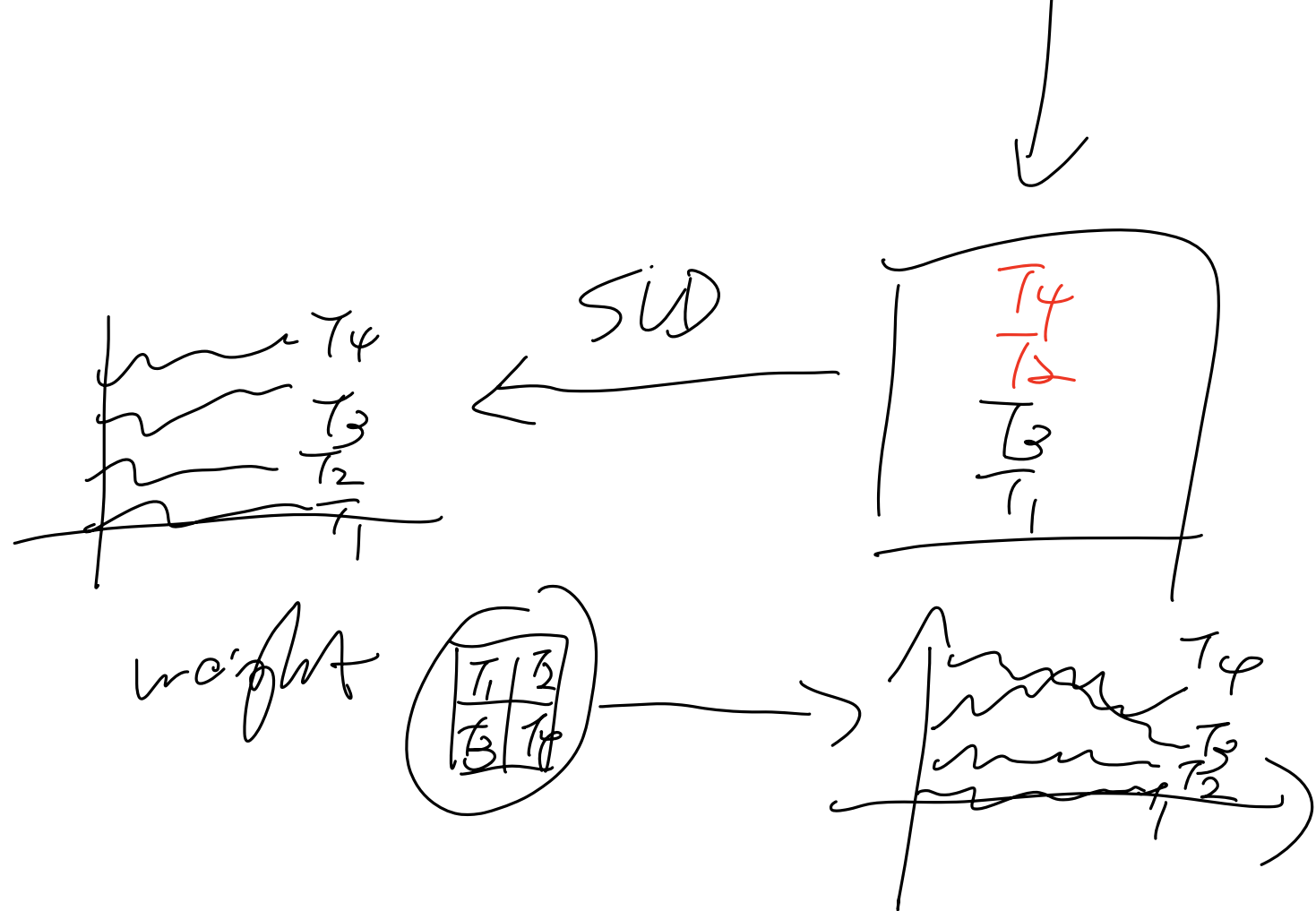
$$A = U \Sigma V^T$$



$$S_L = A^T A = V \Sigma^T U^T U \Sigma V^T$$

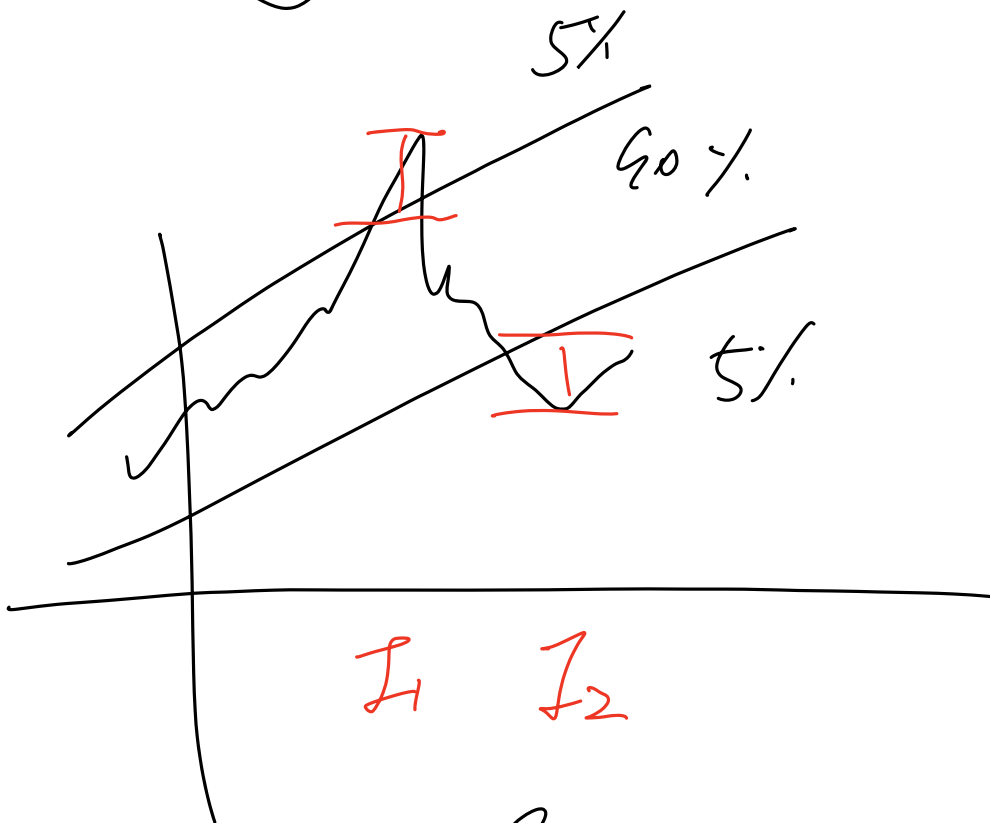
$$S_R = A A^T = U \Sigma V^T V \Sigma^T U^T$$





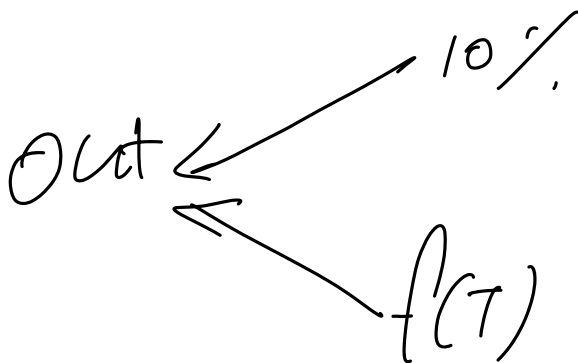
- ① weights to Topic
  - ② Reorder
-

# Matching



$I = \text{counts of best} \times \text{out value}$

$$f(T) = \sum_{\text{sid}} (\text{Evalor}(T))$$



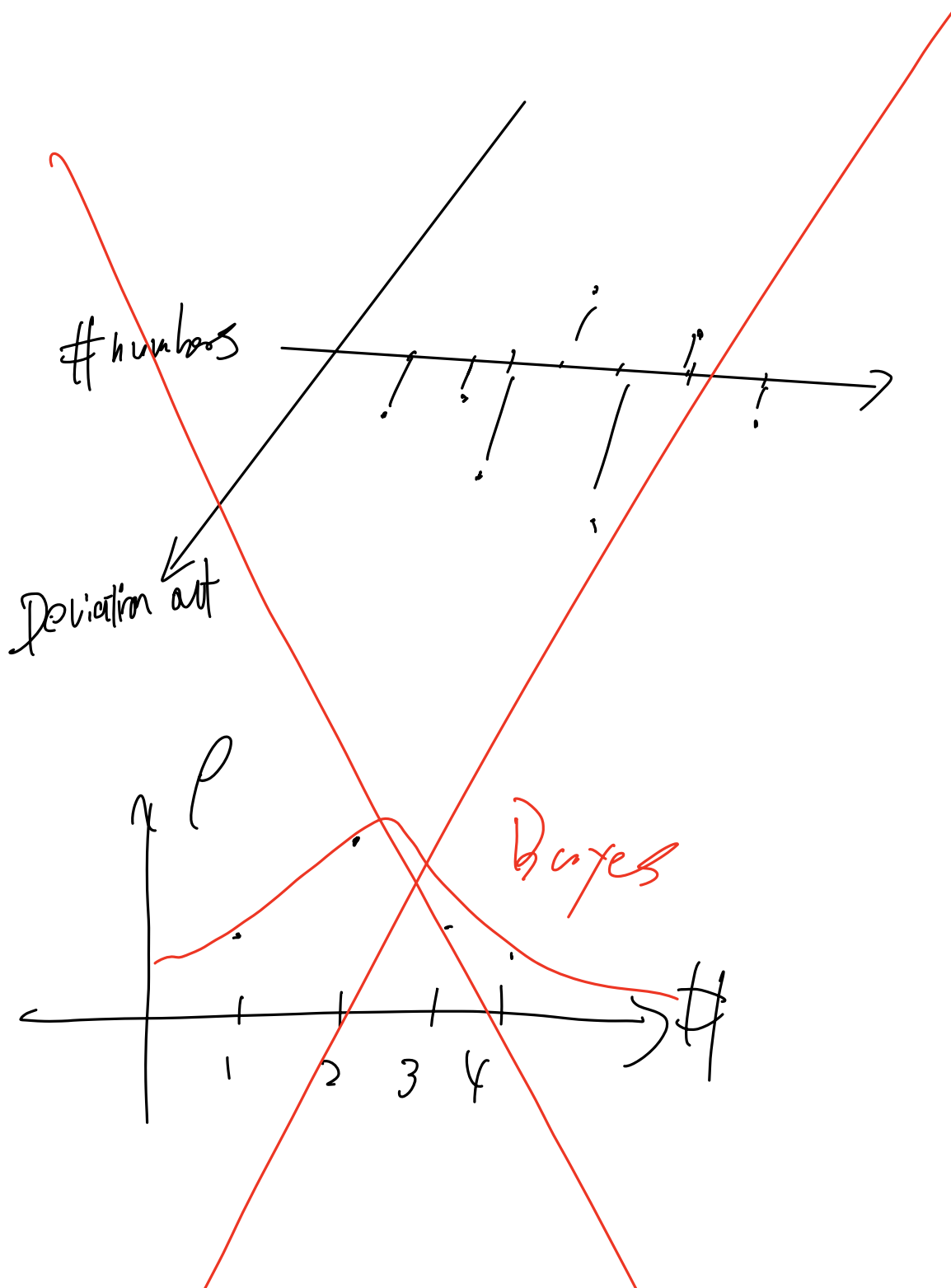
If no  $f(T)$ ,

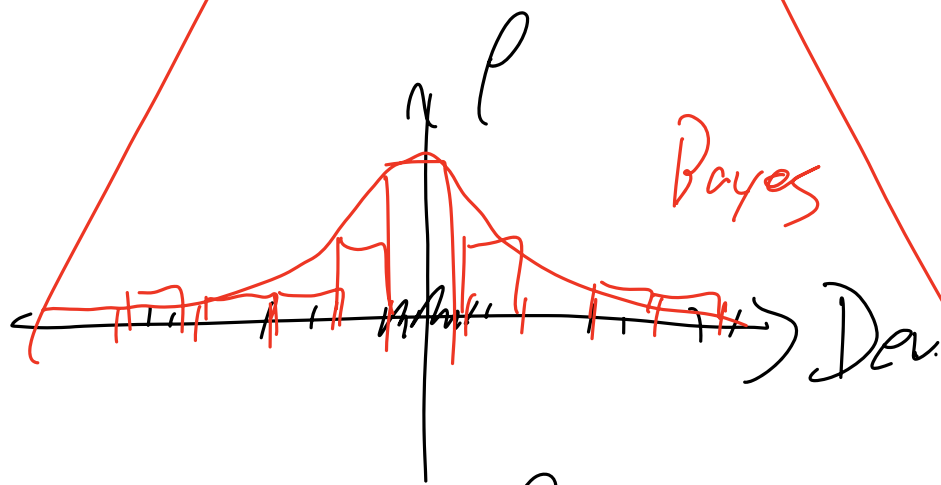
LLN gives 90 in 10 out on avg.

so, if we sampled in history data

80 in 20 out

we have 10 out control towards  $f'(T)$





Training for fitting

$$f(\tau) \xrightarrow{\text{out}} P(t, f(t_x))$$

10%  $\nearrow$

Important Scores  $\{I_i\}$

$$df_t = \left( \sum_{i=1}^N \alpha_i I_i \right) b(t, \theta) dt + \left( \sum_{i=1}^N \beta_i I_i \right) \sigma(b, \theta) dW_t$$

$$\text{Let } b(f(\tau), t, \theta) = \left( \sum_{i=1}^N \alpha_i I_i \right) b$$

$$\sigma(f(\tau), t, \theta) = \left( \sum_{i=1}^N \beta_i I_i \right) \sigma$$

Params  
 $\alpha_i ; \beta_i$  train

Then

$$dP_t = \tilde{\mu} dt + \tilde{\sigma} dW_t$$

Constant Initial  
Stone-Weierstrass

GBM

$$dP_t = \tilde{\mu} S_t dt + \tilde{\sigma} S_t dW_t$$

guarantees  
Approximation

Then, Fokker-Planck is

$$\begin{cases} \partial_t P = L^* P & (t > 0) \\ P(t, 0) = P_0(x) \end{cases}$$

$$L^* g := -\nabla(\tilde{\mu} S g(S)) + \nabla^2 : (a(S, t) g(S))$$

$$\text{with } \nabla^2 : (a g) = \sum_{i,j} \partial_{ij} (a_{ij} g) = \text{tr}(H_{a g})$$

$$a = \frac{1}{2} \sigma \sigma^T = \frac{1}{2} \tilde{\sigma}^2 S^2$$

$$\partial_t P = L^* P = -\partial_x(\tilde{\mu} S g(S)) + \partial_x^2 \left( \frac{1}{2} \tilde{\sigma}^2 S^2 g(S) \right)$$

$$P = \int \mathcal{P}(x, t | y, 0) p_0(y) dy$$

Transition Dens.?

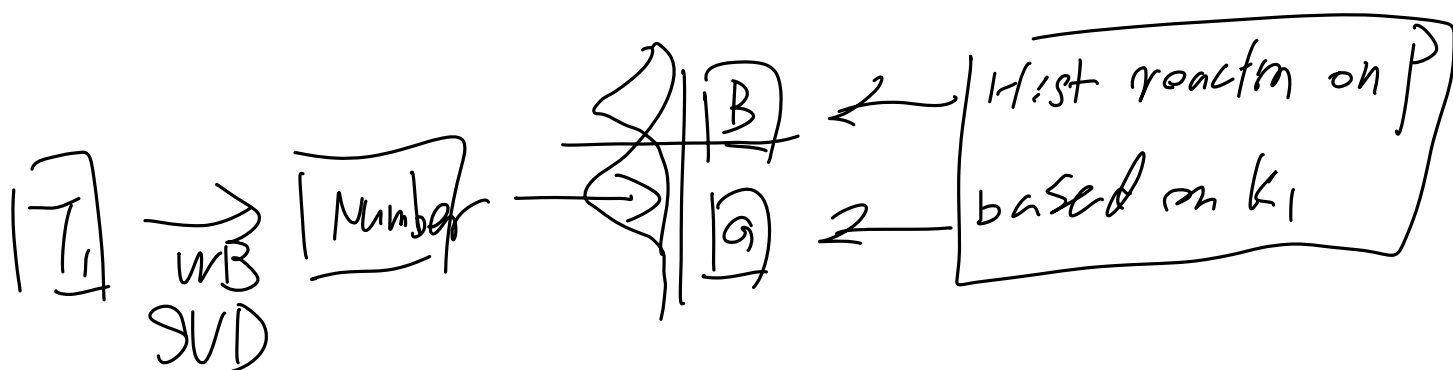
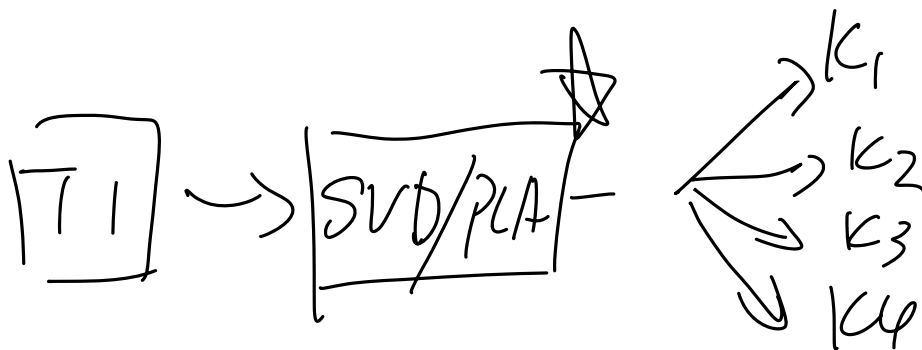
$$\left\{ \begin{array}{l} \partial_t \mathcal{P} = \mathcal{L}_S^* \mathcal{P} \\ \mathcal{P}(x, s | y, s) = \delta(x - y) \end{array} \right., \quad \mathcal{P}(x, t_i | y, t_0)$$

transition density

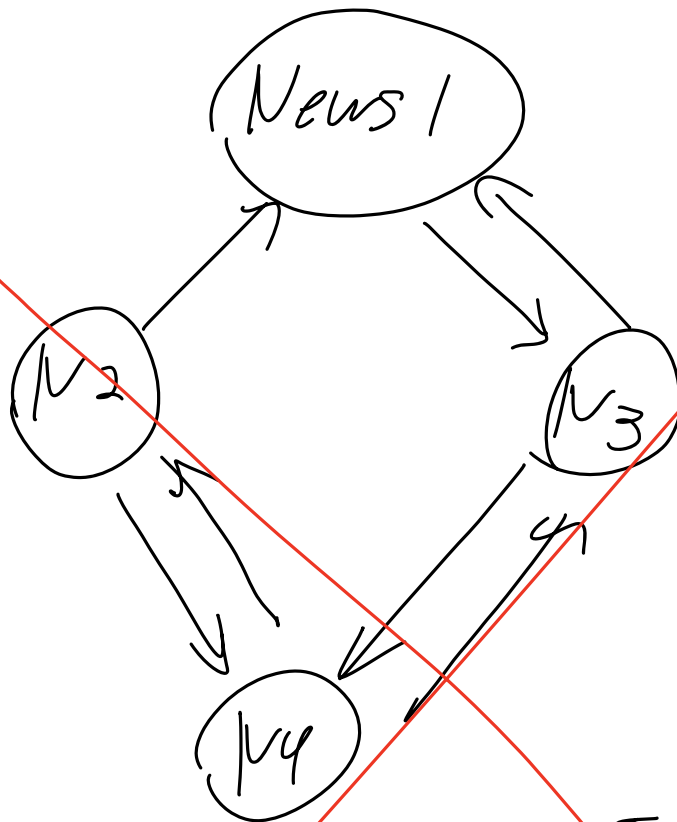
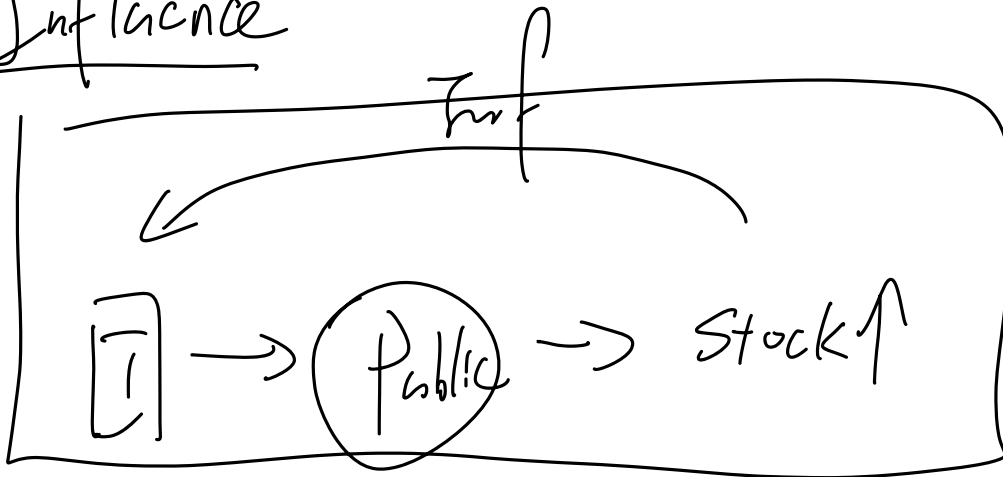
loss function

$$\| \partial_t \mathcal{P} - \mathcal{L}_S^* \mathcal{P} \|$$


---



# Influence



$$\Rightarrow \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \text{ Matrix}$$

$$\Rightarrow V = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix}$$

$$E_{\text{norm}} / \cap = 1$$

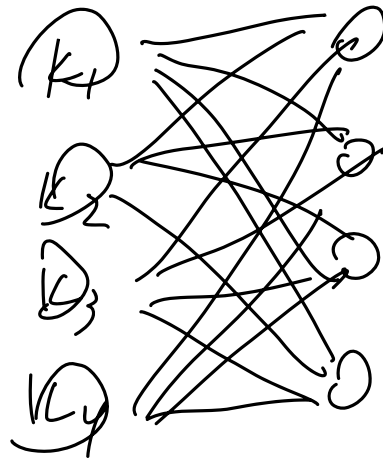
$$r_i = \frac{\theta_i}{\sum_i \theta_i}$$

Possible Factor: Fast, Huge

or use ML to see more general  
factors for agents

Like MN

random data



⋮

Goal: Effectiveness

